



here are the notes add details to each and every concept heading on this and add concepts that u feel are lacking (but mark them as additional) get to work do not miss anything

I'll now create a comprehensive, detailed expansion of your Linux command-line notes with additional concepts marked accordingly.

Linux Command Line Full Course - COMPREHENSIVE EXPANDED NOTES

Based on the 7 hour 57 minute tutorial by ProgrammingKnowledge (Feb 6, 2024), expanded with additional cybersecurity-focused concepts and advanced topics.^{[1][2][3]}

1. Installation & Setup (0:00:01 - 0:24:36)

VirtualBox Installation

Purpose: Virtualization software that allows running different operating systems (guest OS) on your main host operating system without dual-booting. Creates isolated virtual machines (VMs) that share hardware resources.^[^1]

Why VirtualBox for Cybersecurity:

- Safe sandbox environment for testing malware and exploits
- Ability to create snapshots before system modifications
- Network isolation prevents accidental damage to host system
- Practice penetration testing without risking real infrastructure
- Multiple OS environments for cross-platform security testing^[^1]

Installation Steps:

1. Download VirtualBox from [virtualbox.org](https://www.virtualbox.org) (verify GPG signature for security)
2. Download Ubuntu Desktop ISO (~3.4 GB) from official Ubuntu website
3. Verify ISO checksum (SHA256) to ensure file integrity
4. Create new virtual machine with descriptive name
5. Configure VM settings:

- **RAM:** Minimum 4GB (8GB recommended for security tools like Wireshark, Metasploit)
- **Virtual hard disk:** Dynamically allocated, 50GB minimum (100GB for penetration testing labs)
- **Video memory:** 128 MB with 3D acceleration enabled
- **Network adapter:** Configure NAT, Bridged, or Host-Only based on testing needs

6. Attach Ubuntu ISO to virtual optical drive

7. Install Ubuntu following on-screen prompts

8. Install VirtualBox Guest Additions for enhanced features (shared clipboard, drag-and-drop, better resolution)[^1]

Post-Installation Setup

Basic Configuration:

- **Default login:** Username and password set during installation
- **Full-screen mode:** Right Control + F (or View menu → Full-screen Mode)
- **Terminal shortcut:** Control + Alt + T (most important shortcut for command-line work)
- **Power off command:** `poweroff` or `sudo shutdown -h now`
- **Reboot command:** `reboot` or `sudo reboot`[^1]

ADDITIONAL: System Update After Installation

```
sudo apt update           # Update package lists
sudo apt upgrade -y       # Upgrade all packages
sudo apt dist-upgrade     # Upgrade distribution packages
sudo apt autoremove       # Remove unnecessary packages
```

Updates ensure you have latest security patches and bug fixes.[^4]

ADDITIONAL: Snapshot Management

- Create snapshots before major changes
- Name snapshots descriptively: "Fresh Install", "After Updates", "Before Tool Installation"
- Use snapshots to quickly restore system to known-good state
- Practice recovery procedures for incident response training[^1]

2. Basic Navigation Commands (0:24:36 - 0:46:16)

pwd Command - Print Working Directory

Purpose: Displays the absolute path of your current directory location in the filesystem hierarchy.[^1]

Basic Syntax: `pwd [options]`

Common Options:

- `pwd` - Display current directory
- `pwd -L` - Display logical path (follows symbolic links)
- `pwd -P` - Display physical path (resolves symbolic links)

Example Usage:

```
pwd                # Output: /home/username/Documents
cd /usr/local/bin
pwd                # Output: /usr/local/bin
```

Why It Matters: Prevents accidental file operations in wrong directories, essential for scripting and automation.[^1]

ls Command - List Directory Contents

Purpose: Lists files and directories in specified location with various formatting and filtering options.[^1]

Basic Syntax: `ls [options] [directory]`

Common Options:

- `ls` - List files and directories in current location
- `ls -l` - Long format (permissions, owner, group, size, modification date)
- `ls -h` - Human-readable file sizes (with `-l`) - shows KB, MB, GB instead of bytes
- `ls -a` - Show hidden files (files starting with `.`)
- `ls -lh` - Combine long format with human-readable sizes
- `ls -la` - Long format including hidden files
- `ls -ld [directory]` - Show details of specific directory itself, not contents
- `ls -lt` - Sort by modification time (newest first)
- `ls -ltr` - Sort by modification time (oldest first) - **useful for forensics**
- `ls -ls` - Sort by file size (largest first)
- `ls -R` - Recursive listing (show subdirectories)

Examples:

Understanding Long Format Output:

ADDITIONAL: Color Coding in ls Output

- ## cd Command - Change Directory

Basic Syntax: `cd [path]`

Path Types:

- **Absolute path:** Full path from root (e.g., /home/user/Documents)
- **Relative path:** Path from current location (e.g., Documents or ../Downloads)

Common Uses:

```
cd ../../      # Move up two directory levels
cd ~           # Go directly to home directory
cd             # Also goes to home directory
cd -           # Go to previous directory (toggle between two locations)
cd /           # Go to root directory
```

Key Concepts:

- **Tab completion:** Press Tab to auto-complete directory names
- **Case sensitivity:** Linux is case-sensitive (Desktop ≠ desktop ≠ DESKTOP)
- **Special characters:** Use quotes or backslash for spaces: `cd "My Documents"` or `cd My\Documents`
- **Tilde (~):** Represents home directory (`/home/username`)
- **Dot (.):** Represents current directory
- **Double dot (..):** Represents parent directory^[1]

ADDITIONAL: Directory Shortcuts and Navigation

```
cd ~           # Home directory
cd ~/Documents # Documents in home
cd -           # Previous directory (very useful!)
cd /           # Root directory
cd /tmp        # Temporary directory
cd /var/log    # System log directory
cd /etc        # System configuration directory
```

ADDITIONAL: File System Hierarchy Standard (FHS)

Understanding Linux Directory Structure:

The Linux filesystem follows a tree-like hierarchical structure governed by the Filesystem Hierarchy Standard (FHS).^{[5][2]}

Key Directories:

/ (Root Directory)

- Top of filesystem hierarchy
- All directories branch from here
- Never delete or modify contents without understanding impact^{[6][2]}

/bin (Essential Binaries)

- Essential command-line binaries for all users
- Contains: `ls`, `cp`, `mv`, `cat`, `chmod`, `bash`, etc.
- Required for system boot and single-user mode^{[7][2]}

/sbin (System Binaries)

- System administration binaries
- Contains: fdisk, fsck, iptables, reboot, shutdown
- Typically requires root privileges^[2]

/etc (Configuration Files)

- System-wide configuration files
- Key files:
 - `/etc/passwd` - User account information
 - `/etc/shadow` - Encrypted passwords (cybersecurity critical!)
 - `/etc/group` - Group information
 - `/etc/fstab` - Filesystem mount table
 - `/etc/hosts` - Hostname to IP mappings
 - `/etc/ssh/sshd_config` - SSH server configuration^{[7][2]}

/home (User Home Directories)

- Personal directories for each user
- Format: `/home/username`
- Contains user files, configurations, downloads^[2]

/root (Root User Home)

- Home directory for root/superuser
- Not to be confused with `/` (root directory)
- Contains root's personal files and configurations^{[6][2]}

/var (Variable Data)

- Variable data that changes during system operation
- Subdirectories:
 - `/var/log` - System and application logs (forensics goldmine!)
 - `/var/www` - Web server files
 - `/var/tmp` - Temporary files preserved between reboots
 - `/var/mail` - User mailboxes^{[5][2]}

/tmp (Temporary Files)

- Temporary files created by applications
- Usually cleared on reboot
- World-writable directory (security concern!)
- Use with caution in multi-user environments^{[5][2]}

/usr (User Programs)

- Secondary hierarchy for read-only user data
- Contains most user utilities and applications
- Subdirectories:
 - `/usr/bin` - User commands
 - `/usr/sbin` - Non-essential system binaries
 - `/usr/lib` - Libraries for binaries
 - `/usr/local` - Locally compiled/installed software^{[2][5]}

/opt (Optional Software)

- Third-party software and packages
- Add-on applications from vendors
- Self-contained application packages^[^2]

/dev (Device Files)

- Device files representing hardware
- Examples:
 - `/dev/sda` - First hard drive
 - `/dev/null` - Null device (discards all data)
 - `/dev/random` - Random number generator
 - `/dev/tty` - Terminal devices^{[7][2]}

/proc (Process Information)

- Virtual filesystem for kernel and process information
- Not stored on disk (created dynamically)
- Contains:
 - `/proc/cpuinfo` - CPU information
 - `/proc/meminfo` - Memory information
 - `/proc/[PID]` - Process-specific information^[^2]

/sys (System Information)

- Virtual filesystem for device and kernel info
- Interface to kernel's view of hardware
- Used for hardware management and monitoring^[^5]

/boot (Boot Files)

- Files needed for system boot
- Contains kernel, bootloader files
- Critical for system startup^[^2]

/lib (Essential Libraries)

- Essential shared libraries for system boot
- Kernel modules stored here
- Required by binaries in /bin and /sbin^[2]

/mnt (Mount Point)

- Temporary mount point for filesystems
- System administrators manually mount drives here^[2]

/media (Removable Media)

- Automatic mount points for removable media
- USB drives, CDs, DVDs mount here
- Modern systems auto-mount here^[2]

Cybersecurity Relevance:

- /etc/shadow - Password security
- /var/log - Incident investigation
- /tmp - Potential malware staging area
- /proc - Process analysis and monitoring
- /dev - Device security and hardware access^{[7][2]}

3. Directory Management (0:46:16 - 1:03:00)

mkdir Command - Make Directory

Purpose: Creates new directories (folders) in the filesystem with various options for structure creation.^[1]

Basic Syntax: `mkdir [options] directory_name`

Common Options:

- `mkdir directory_name` - Create single directory
- `mkdir dir1 dir2 dir3` - Create multiple directories at once
- `mkdir -p parent/child/grandchild` - Create nested directories (parent directories created automatically)
- `mkdir -v directory_name` - Verbose mode (confirms creation with message)
- `mkdir -m 755 directory_name` - Create with specific permissions

Examples:


```
mkdir University          # Single directory
mkdir Students Subjects Workers  # Multiple directories
mkdir -p University/Students/Aayush # Nested structure
mkdir -v Linux            # With confirmation message
mkdir -m 700 PrivateFolder  # Create with rwx----- permissions
```

Important Notes:

- Directory names are case-sensitive
- Without `-p`, parent directories must exist first
- Use quotes for names with spaces: `mkdir "My Folder"`
- Avoid special characters: `/ * ? " < > |` in directory names
- Best practice: Use underscores or hyphens instead of spaces^[1]

ADDITIONAL: Directory Naming Best Practices

```
# Good naming conventions
mkdir project_2025
mkdir web_server_logs
mkdir backup-2025-11-21

# Avoid these
mkdir "My Files!"          # Special characters
mkdir "Project (2025)"     # Parentheses problematic in scripts
mkdir ~/Desktop/Test\ Files # Spaces require escaping
```

rmdir Command - Remove Empty Directory

Purpose: Removes empty directories only (safety feature to prevent accidental data deletion).
^[1]

Basic Syntax: `rmdir [options] directory_name`

Common Options:

- `rmdir directory_name` - Remove empty directory
- `rmdir -p dir1/dir2/dir3` - Remove nested empty directories
- `rmdir -v directory_name` - Verbose mode (show what's being deleted)

Examples:

```
rmdir test_directory      # Remove empty directory
rmdir -p dir1/dir2/dir3   # Remove nested empty directories
rmdir -v old_folder       # Remove with confirmation message
```

Limitations:

- Only removes empty directories

- For non-empty directories, use `rm -r` (be extremely careful!)
- Use `ls -la directory_name` first to verify it's empty^[1]

ADDITIONAL: Safe Directory Removal Workflow

```
# 1. Check if directory is empty
ls -la test_dir

# 2. If empty, use rmdir (safest)
rmdir test_dir

# 3. If not empty and you're SURE you want to delete
ls -R test_dir          # Review contents recursively
rm -ri test_dir         # Interactive removal (asks for each file)
# OR for non-interactive (DANGEROUS!)
rm -rf test_dir         # Force removal without confirmation
```

ADDITIONAL: Tree Command for Directory Visualization

```
sudo apt install tree  # Install tree command
tree -L 2 /home        # Show 2 levels of directory structure
tree -d               # Show directories only
tree -a               # Show hidden files
tree -h               # Show file sizes in human-readable format
tree -p               # Show permissions
```

4. File Operations (1:12:40 - 1:51:18)

touch Command - Create Files

Purpose: Creates new empty files or updates timestamps of existing files.^[1]

Basic Syntax: `touch [options] filename`

Common Uses:

```
touch file.txt          # Create empty file
touch file1.txt file2.txt # Create multiple files
touch -a file.txt       # Update access time only
touch -m file.txt       # Update modification time only
touch -r ref.txt file.txt # Copy timestamp from ref.txt to file.txt
touch -t 202511211200 file.txt # Set specific timestamp (YYYYMMDDhhmm)
touch -d "2 days ago" file.txt # Set timestamp to 2 days ago
```

Key Points:

- Creates file if it doesn't exist
- Updates timestamp if file exists (both access and modification times)

- Works with multiple files simultaneously
- Useful for testing file operations
- Commonly used in scripts to create log files[^1]

Cybersecurity Applications:

- Create dummy files for testing
- Reset file timestamps (anti-forensics technique - ethical use only!)
- Generate test files for permission testing
- Create placeholder files in scripts[^1]

ADDITIONAL: Advanced touch Usage

```
# Create files with specific extensions
touch script.sh config.conf log.txt

# Create numbered files
touch file{1..10}.txt      # Creates file1.txt through file10.txt

# Create files in different directories
touch ~/Documents/report.txt /tmp/temp.log

# Create file and set permission in one line
touch secure.txt && chmod 600 secure.txt
```

cat Command - Concatenate and Display Files

Purpose: Displays file contents, creates files with content, concatenates multiple files.[^1]

Basic Syntax: `cat [options] [file(s)]`

Common Operations:

```
cat > file.txt          # Create file with content (Ctrl+D to save)
cat file.txt            # Display file contents
cat file1.txt file2.txt # Display multiple files sequentially
cat -n file.txt         # Display with line numbers
cat -b file.txt         # Number non-blank lines only
cat -T file.txt         # Show tabs as ^I
cat -E file.txt         # Show $ at end of each line
cat -A file.txt         # Show all non-printing characters
cat file1.txt >> file2.txt # Append file1 to file2
```

Practical Examples:

```
# Create file with content
cat > myfile.txt
This is line 1
This is line 2
[Press Ctrl+D to save]
```

```
# Combine multiple files
cat file1.txt file2.txt > combined.txt

# Display with line numbers (useful for code review)
cat -n script.sh

# Append to existing file
cat additional.txt >> main.txt

# Create multi-line configuration file
cat > config.conf << EOF
server_name=localhost
port=8080
debug=true
EOF
```

ADDITIONAL: Cat Command Alternatives for Large Files

```
# For large files, use these instead
less file.txt          # Paginated view (press q to quit)
more file.txt          # Simple pager (space to continue)
head -n 20 file.txt    # First 20 lines
tail -n 20 file.txt    # Last 20 lines

# For binary files, use
xxd file.bin           # Hexadecimal view
strings file.bin       # Extract readable strings
```

rm Command - Remove Files

Purpose: Deletes files and directories (PERMANENT operation - no Recycle Bin in Linux!).^[1]

Basic Syntax: `rm [options] file/directory`

Common Options:

- `rm file.txt` - Remove single file
- `rm -i file.txt` - Interactive mode (asks confirmation before deletion)
- `rm -r directory` - Remove directory and contents recursively
- `rm -f file.txt` - Force removal (no confirmation, ignores non-existent files)
- `rm -rf directory` - Force remove directory recursively (EXTREMELY DANGEROUS!)
- `rm -v file.txt` - Verbose mode (shows what's being deleted)
- `rm *.txt` - Remove all .txt files in current directory

Examples:

```
rm test.txt          # Remove single file
rm *.txt             # Remove all .txt files
rm -i important.txt  # Ask before removing
```

```
rm -r old_folder      # Remove directory and contents
rm -rf temp/*         # Remove all contents (BE CAREFUL!)
rm -v *.log           # Remove all log files with confirmation
```

Safety Tips:

- **ALWAYS** use `-i` flag when unsure: `rm -i file.txt`
- **NEVER** use `rm -rf /` (destroys entire system!)
- Double-check before using wildcards (`*`, `?`)
- Use `ls` first to preview what will be deleted: `ls *.txt` before `rm *.txt`
- For important files, move to trash folder instead: `mv file.txt ~/.local/share/Trash/`
- Create aliases for safety: `alias rm='rm -i'[^1]`

ADDITIONAL: Secure File Deletion

```
# Standard rm leaves recoverable data
rm file.txt

# Secure deletion (overwrite before deletion)
shred -vzf -n 3 sensitive.txt  # 3-pass overwrite
shred -vzf -n 7 top_secret.txt # 7-pass overwrite (DoD standard)

# Wipe entire directory securely
sudo apt install wipe
wipe -rf sensitive_folder

# For SSDs, use TRIM
fstrim -v /                  # TRIM entire filesystem
```

ADDITIONAL: File Recovery Prevention

Understanding that `rm` doesn't actually delete data immediately is crucial for cybersecurity:

- File content remains on disk until overwritten
- Forensic tools can recover deleted files
- Use `shred` for sensitive data deletion
- Encrypt sensitive files before deletion for extra security[^1]

cp Command - Copy Files

Purpose: Creates copies of files and directories, preserving or modifying attributes.[^1]

Basic Syntax: `cp [options] source destination`

Common Options:

- `cp file.txt backup.txt` - Copy single file
- `cp -i file.txt dest.txt` - Interactive (ask before overwrite)

- `cp -r directory backup_dir` - Copy directory recursively
- `cp -v file.txt backup.txt` - Verbose mode
- `cp -p file.txt backup.txt` - Preserve attributes (permissions, timestamps, ownership)
- `cp -u source.txt dest.txt` - Update only (copy if source is newer)
- `cp -a directory backup` - Archive mode (preserves everything, implies -r)
- `cp file1.txt file2.txt dir/` - Copy multiple files to directory

Examples:

```
# Copy single file
cp document.txt document_backup.txt

# Copy directory recursively
cp -r Projects Projects_Backup

# Copy with confirmation
cp -i important.txt important_new.txt

# Preserve all attributes
cp -p config.conf config.conf.bak

# Copy multiple files to directory
cp file1.txt file2.txt file3.txt ~/Documents/

# Archive mode (best for backups)
cp -a /home/user/data /backup/data
```

ADDITIONAL: Advanced Copy Operations

```
# Copy only if source is newer
cp -u updated.txt /backup/

# Copy and create backup of destination
cp --backup=numbered file.txt dest.txt # Creates dest.txt.~1~, dest.txt.~2~, etc.

# Copy with progress (using rsync)
rsync -ah --progress source.txt dest.txt

# Copy while preserving sparse files
cp --sparse=always large_file.img backup.img

# Copy symlinks as symlinks (not the target)
cp -P symlink /destination/
```

mv Command - Move/Rename Files

Purpose: Moves files/directories to new location or renames them (atomic operation).[^1]

Basic Syntax: `mv [options] source destination`

Common Options:

- `mv oldname.txt newname.txt` - Rename file in same directory
- `mv file.txt ~/Documents/` - Move file to different directory
- `mv -i file.txt dest.txt` - Ask before overwriting existing file
- `mv -v file.txt newlocation/` - Verbose mode
- `mv -u source.txt dest.txt` - Update only (move if source is newer)
- `mv -n file.txt dest.txt` - No clobber (don't overwrite existing files)
- `mv *.txt TextFiles/` - Move all .txt files to directory

Examples:

```
# Rename file
mv report.txt final_report.txt

# Move file to different directory
mv photo.jpg ~/Pictures/

# Move and rename simultaneously
mv temp.txt ~/Documents/archive.txt

# Move multiple files
mv file1.txt file2.txt file3.txt ~/Backup/

# Move directory
mv old_project_name new_project_name

# Safe move (interactive)
mv -i important.txt /destination/
```

ADDITIONAL: Bulk Rename Operations

```
# Rename with pattern (requires rename utility)
sudo apt install rename
rename 's/old/new/' *.txt          # Replace 'old' with 'new' in all .txt files
rename 's/ /_/g' *                 # Replace spaces with underscores
rename 'y/A-Z/a-z/' *              # Convert uppercase to lowercase

# Batch rename with loop
for file in *.jpg; do
    mv "$file" "vacation_${file}"
done

# Add date prefix to files
for file in *.log; do
```

```
mv "$file" "$(date +%Y%m%d)_${file}"
done
```

ADDITIONAL: File Viewing Commands

These are essential for quickly inspecting file contents without opening editors.^{[3][1]}

head Command - View First Lines

Basic Syntax: `head [options] file`

Common Uses:

```
head file.txt           # First 10 lines (default)
head -n 5 file.txt      # First 5 lines
head -n 20 file.txt     # First 20 lines
head -c 100 file.txt    # First 100 bytes
head -n -10 file.txt    # All except last 10 lines
```

tail Command - View Last Lines

Basic Syntax: `tail [options] file`

Common Uses:

```
tail file.txt           # Last 10 lines (default)
tail -n 5 file.txt      # Last 5 lines
tail -n 20 file.txt     # Last 20 lines
tail -f logfile.txt     # Follow file in real-time (essential for log monitoring!)
tail -F logfile.txt     # Follow with retry (handles log rotation)
tail -n +10 file.txt    # From line 10 to end
```

Practical Use Cases:

```
# Monitor system log in real-time (cybersecurity essential!)
tail -f /var/log/syslog

# Monitor Apache access log
tail -f /var/log/apache2/access.log

# View last 50 lines of auth log (failed login attempts)
tail -n 50 /var/log/auth.log

# Monitor multiple logs simultaneously
tail -f /var/log/syslog /var/log/auth.log

# Follow log and grep for errors
tail -f application.log | grep -i error
```


nl Command - Number Lines

Basic Syntax: `nl [options] file`

Common Uses:

```
nl file.txt          # Number all non-empty lines
nl -ba file.txt      # Number all lines including blank
nl -w3 file.txt      # Width of 3 characters for numbers
nl -s ' . ' file.txt # Custom separator (line number. content)
```

ADDITIONAL: Text Processing Commands

grep Command - Search Text Patterns

Purpose: Searches for patterns in files using regular expressions (most powerful text search tool).^{[8][3]}

Basic Syntax: `grep [options] pattern [file]`

Common Options:

```
grep "pattern" file.txt      # Search for pattern in file
grep -i "pattern" file.txt   # Case-insensitive search
grep -r "pattern" /directory # Recursive search in directory
grep -n "pattern" file.txt   # Show line numbers
grep -v "pattern" file.txt   # Invert match (show lines NOT matching)
grep -c "pattern" file.txt   # Count matching lines
grep -l "pattern" *.txt      # Show only filenames with matches
grep -w "word" file.txt      # Match whole word only
grep -A 3 "pattern" file.txt # Show 3 lines After match
grep -B 3 "pattern" file.txt # Show 3 lines Before match
grep -C 3 "pattern" file.txt # Show 3 lines Context (before and after)
```

Cybersecurity Applications:

```
# Find failed login attempts
grep "Failed password" /var/log/auth.log

# Find IP addresses in log
grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" access.log

# Search for SUID files (privilege escalation)
find / -perm -4000 2>/dev/null | grep -i "binary_name"

# Find open ports
netstat -tulpn | grep LISTEN

# Search for suspicious processes
ps aux | grep -E "(nc|netcat|nmap)"
```

find Command - Search Files and Directories

Purpose: Searches for files and directories based on various criteria.[^1]

Basic Syntax: `find [path] [options] [expression]`

Common Uses:

```
find . -name "file.txt"           # Find by exact name
find . -iname "*.txt"             # Case-insensitive name search
find . -type f                   # Find files only
find . -type d                   # Find directories only
find . -size +10M                # Files larger than 10MB
find . -mtime -7                 # Modified in last 7 days
find . -atime +30                # Accessed more than 30 days ago
find . -perm 777                 # Find files with specific permissions
find . -user username            # Find files by owner
find . -empty                    # Find empty files/directories
```

Advanced Find Operations:

```
# Find and execute command
find . -name "*.log" -exec rm {} \;           # Delete all log files
find . -type f -name "*.txt" -exec wc -l {} \; # Count lines in txt files

# Find SUID binaries (security audit)
find / -perm -4000 -type f 2>/dev/null

# Find world-writable files (security risk)
find / -perm -002 -type f 2>/dev/null

# Find recently modified config files
find /etc -name "*.conf" -mtime -1

# Find large files consuming disk space
find / -type f -size +100M 2>/dev/null
```

5. File Compression (2:10:21 - 2:34:43)

zip Command - Compress Files

Purpose: Compresses files and directories into .zip archives for storage or transfer.[^1]

Basic Syntax: `zip [options] archive.zip file(s)`

Common Options:

- `zip archive.zip file.txt` - Compress single file
- `zip -r folder.zip directory/` - Compress directory recursively
- `zip -u archive.zip newfile.txt` - Update archive (add/replace files)

- `zip -m archive.zip file.txt` - Move to archive (delete original after compression)
- `zip -e secure.zip file.txt` - Encrypt with password
- `zip -9 archive.zip file.txt` - Maximum compression
- `zip -1 archive.zip file.txt` - Fast compression (lower ratio)
- `zip -q archive.zip file.txt` - Quiet mode (suppress output)

Examples:

```
# Compress single file
zip backup.zip document.txt

# Compress directory recursively
zip -r project.zip ProjectFolder/

# Password-protected archive (cybersecurity!)
zip -e confidential.zip secret.txt
# Will prompt for password twice

# Maximum compression
zip -9 -r compressed.zip large_directory/

# Exclude specific files
zip -r archive.zip directory/ -x "*.log" "*.tmp"

# Add to existing archive
zip -u existing.zip newfile.txt
```

ADDITIONAL: Advanced Compression Options

```
# Encrypt with AES-256 (stronger encryption)
zip -e -r secure.zip sensitive_data/

# Split large archive into parts
zip -r -s 100m large.zip huge_folder/ # 100MB parts

# Test archive integrity
zip -T archive.zip

# List archive contents without extracting
zip -sf archive.zip

# Create archive and delete originals
zip -rm cleanup.zip *.tmp
```

unzip Command - Extract Files

Purpose: Extracts files from .zip archives.[^1]

Basic Syntax: `unzip [options] archive.zip`

Common Options:

- `unzip archive.zip` - Extract to current directory
- `unzip -d /path/to/dir archive.zip` - Extract to specific directory
- `unzip -l archive.zip` - List contents without extracting
- `unzip -t archive.zip` - Test archive integrity
- `unzip -q archive.zip` - Quiet mode (no output)
- `unzip -o archive.zip` - Overwrite files without prompting
- `unzip -n archive.zip` - Never overwrite existing files
- `unzip -P password archive.zip` - Extract password-protected archive

Examples:

```
# Extract archive
unzip photos.zip

# Extract to specific location
unzip -d ~/Downloads backup.zip

# View contents first (always recommended!)
unzip -l data.zip

# Test archive before extraction
unzip -t archive.zip

# Extract specific file only
unzip archive.zip specific_file.txt

# Extract password-protected archive
unzip -P mypassword secure.zip
```

ADDITIONAL: tar Command - Tape Archive

Purpose: Archives files (with or without compression) - the Unix/Linux standard archiving tool.
[^3]

Basic Syntax: `tar [options] archive.tar file(s)`

Common Operations:

```
# Create archive (no compression)
tar -cvf archive.tar directory/

# Create compressed archive (gzip)
tar -czvf archive.tar.gz directory/

# Create compressed archive (bzip2 - better compression)
tar -cjvf archive.tar.bz2 directory/

# Create compressed archive (xz - best compression)
tar -cJvf archive.tar.xz directory/
```

```
# Extract archive
tar -xvf archive.tar

# Extract compressed archive (gzip)
tar -xzvf archive.tar.gz

# Extract to specific directory
tar -xzvf archive.tar.gz -C /destination/path

# List archive contents
tar -tvf archive.tar

# Append files to existing archive
tar -rvf archive.tar newfile.txt

# Extract specific file
tar -xzvf archive.tar.gz path/to/specific/file
```

Option Meanings:

- **c** - Create archive
- **x** - Extract archive
- **t** - List contents
- **v** - Verbose (show progress)
- **f** - File (specify archive name)
- **z** - Gzip compression (.tar.gz)
- **j** - Bzip2 compression (.tar.bz2)
- **J** - XZ compression (.tar.xz)

Cybersecurity Applications:

```
# Backup system configuration
tar -czvf config_backup_$(date +%Y%m%d).tar.gz /etc/

# Create evidence archive with timestamps
tar -czvf evidence_$(date +%Y%m%d_%H%M%S).tar.gz /evidence/

# Backup with permissions preserved
tar -czvpf backup.tar.gz /important/data

# Exclude sensitive files from archive
tar -czvf backup.tar.gz --exclude='*.key' --exclude='*.pem' /data/
```

Full permission: 754

File Type Indicators:

- - Regular file
- d Directory
- l Symbolic link
- c Character device
- b Block device
- p Named pipe
- s Socket[^1]

chmod Command - Change Permissions

Purpose: Modifies file and directory permissions using numeric or symbolic modes.[^1]

Basic Syntax: `chmod [options] permissions file`

Numeric Mode (Octal):

```
chmod 755 file.txt      # rwxr-xr-x (owner: all, group/others: read+execute)
chmod 644 file.txt      # rw-r--r-- (owner: read+write, others: read only)
chmod 777 file.txt      # rwxrwxrwx (full access for everyone - DANGEROUS!)
chmod 600 file.txt      # rw----- (owner only access - secure for private files)
chmod 700 directory    # rwx----- (owner full access to directory, others none)
chmod 755 script.sh     # rwxr-xr-x (common for executable scripts)
chmod 400 key.pem       # r----- (read-only for owner - SSH keys)
```

Common Permission Values:

- **777** - Full access for everyone (never use in production!)
- **755** - Owner: full, Group/Others: read+execute (common for programs)
- **700** - Owner only access (secure for private directories)
- **644** - Owner: read+write, Group/Others: read (common for files)
- **600** - Owner only read+write (secure for configuration files)
- **400** - Owner read-only (SSL certificates, SSH keys)

Symbolic Mode:

```
chmod u+x file.txt      # Add execute permission for user
chmod g-w file.txt      # Remove write permission from group
chmod o+r file.txt      # Add read permission for others
chmod a+x script.sh     # Add execute permission for all
chmod u=rwx,g=rx,o=r file.txt # Set exact permissions
chmod -R 755 directory  # Recursive permission change
```

Symbolic Mode Characters:

- **u** = user (owner)
- **g** = group
- **o** = others
- **a** = all (user + group + others)
- **+** = add permission
- **-** = remove permission
- **=** = set exact permission[^1]

Examples:

```
# Make script executable
chmod +x script.sh

# Secure sensitive file
chmod 600 passwords.txt

# Set directory permissions recursively
chmod -R 755 /var/www/html

# Remove all permissions for others
chmod o-rwx private_file.txt

# Set SUID bit (run as owner)
chmod u+s executable

# Set sticky bit on directory (only owner can delete files)
chmod +t /tmp
```

ADDITIONAL: Special Permissions

SUID (Set User ID) - 4xxx

```
chmod 4755 executable # or chmod u+s executable
# File executes with permissions of owner, not executor
# Security risk if not properly managed!
# Example: /usr/bin/passwd (allows users to change passwords)
```

SGID (Set Group ID) - 2xxx

```
chmod 2755 directory # or chmod g+s directory
# On directories: New files inherit group of directory
# On executables: Runs with group permissions
```


Sticky Bit - 1xxx

```
chmod 1777 /shared # or chmod +t /shared
# Only file owner can delete their own files
# Common on /tmp directory
```

Finding Special Permissions:

```
# Find SUID files (potential privilege escalation vectors)
find / -perm -4000 -type f 2>/dev/null

# Find SGID files
find / -perm -2000 -type f 2>/dev/null

# Find sticky bit directories
find / -perm -1000 -type d 2>/dev/null
```

chown Command - Change Ownership

Purpose: Changes the owner and/or group of files and directories (requires root privileges).^[^1]

Basic Syntax: `chown [options] user:group file`

Common Uses:

```
chown john file.txt           # Change owner to john
chown john:developers file.txt # Change owner and group
chown -R john:developers dir/  # Recursive for directories
chown :developers file.txt     # Change group only (same as chgrp)
chown -v john file.txt         # Verbose mode
```

Examples:

```
# Change owner of file
sudo chown alice report.txt

# Change owner and group
sudo chown alice:staff report.txt

# Recursive ownership change
sudo chown -R www-data:www-data /var/www/html

# Change owner, preserve group
sudo chown bob file.txt

# Copy ownership from reference file
sudo chown --reference=file1.txt file2.txt
```

chgrp Command - Change Group

Purpose: Changes only the group ownership of files and directories.[^1]

Basic Syntax: `chgrp [options] group file`

Common Uses:

```
chgrp developers project.txt
chgrp -R staff /shared/folder
chgrp -v wheel executable
```

Examples:

```
# Change group
sudo chgrp developers source_code.c

# Recursive group change
sudo chgrp -R www-data /var/www

# Copy group from reference
sudo chgrp --reference=file1 file2
```

ADDITIONAL: Permission Security Best Practices

Secure File Permission Guidelines:

```
# Home directories
chmod 700 /home/username          # Only owner access

# SSH keys
chmod 700 ~/.ssh                  # SSH directory
chmod 600 ~/.ssh/id_rsa           # Private key
chmod 644 ~/.ssh/id_rsa.pub       # Public key
chmod 644 ~/.ssh/authorized_keys  # Authorized keys
chmod 644 ~/.ssh/known_hosts      # Known hosts

# Web server files
chmod 755 /var/www/html           # Directories
chmod 644 /var/www/html/*.html    # HTML files
chmod 600 /var/www/html/config.php # Config files

# Scripts
chmod 750 /usr/local/bin/script.sh # Executable for owner/group

# Log files
chmod 640 /var/log/application.log # Owner read/write, group read

# Database files
chmod 600 /var/lib/database/*.db  # Owner only
```

Security Audit Commands:

```
# Find world-writable files (security risk!)
find / -xdev -type f -perm -0002 2>/dev/null

# Find world-writable directories
find / -xdev -type d -perm -0002 2>/dev/null

# Find files without owner
find / -xdev -nouser 2>/dev/null

# Find files without group
find / -xdev -nogroup 2>/dev/null

# Check for files with weird permissions
find /home -type f -perm 777 2>/dev/null
```

7. User Management (3:24:50 - 4:07:36)

useradd/adduser Commands - Create Users

Purpose: Creates new user accounts on the system.[^1]

Basic Syntax: useradd [options] username

Common Options:

```
useradd john                # Create basic user
useradd -m john             # Create with home directory
useradd -m -s /bin/bash john # Specify shell
useradd -u 1500 john        # Specify user ID
useradd -c "John Doe" john  # Add comment/description
useradd -g developers john  # Specify primary group
useradd -G sudo,www-data john # Add to additional groups
useradd -e 2026-12-31 john  # Set expiry date
useradd -d /custom/home john # Custom home directory
```

adduser vs useradd:

- **adduser** - Interactive, user-friendly (Ubuntu/Debian) - automatically creates home directory
- **useradd** - Low-level, requires more manual configuration (all distributions)

Example User Creation Workflow:

```
# Method 1: useradd (manual)
sudo useradd -m -s /bin/bash -c "Alice Smith" alice
sudo passwd alice
sudo usermod -aG sudo alice

# Method 2: adduser (interactive - easier)
```

```
sudo adduser alice
# Prompts for password and user info automatically
```

passwd Command - Change Password

Purpose: Sets or changes user passwords with various policy options.[^1]

Basic Syntax: `passwd [options] [username]`

Common Uses:

```
passwd          # Change own password
passwd john     # Change john's password (requires sudo)
passwd -e john  # Force password change on next login
passwd -l john  # Lock account (disable login)
passwd -u john  # Unlock account
passwd -d john  # Delete password (passwordless login - DANGEROUS!)
passwd -S john  # Show password status
```

Password Policy Options:

```
passwd -n 7 john  # Minimum 7 days before password can be changed
passwd -x 90 john # Maximum 90 days before password must be changed
passwd -w 14 john # Warn 14 days before password expiry
passwd -i 30 john # Lock account 30 days after password expiry
```

Examples:

```
# Set password for new user
sudo passwd alice

# Force password change at next login
sudo passwd -e alice

# Lock compromised account
sudo passwd -l suspicious_user

# Check password status
sudo passwd -S alice
# Output: alice P 11/21/2025 7 90 14 30
# P = Password set, creation date, min, max, warn, inactive days
```

groupadd Command - Create Groups

Purpose: Creates new user groups for organizing users and permissions.[^1]

Basic Syntax: `groupadd [options] groupname`

Common Uses:

```
groupadd developers
groupadd -g 1500 staff      # Specify group ID
groupadd -f developers     # Force creation even if exists
groupadd -r system_group  # Create system group
```

Examples:

```
# Create development group
sudo groupadd developers

# Create group with specific GID
sudo groupadd -g 2000 security_team

# Add user to group
sudo usermod -aG developers alice
```

usermod Command - Modify Users

Purpose: Modifies existing user account properties.[^1]

Basic Syntax: `usermod [options] username`

Common Operations:

```
usermod -aG developers john  # Add to additional group (append)
usermod -l johndoe john     # Change username from john to johndoe
usermod -u 1600 john        # Change user ID
usermod -g newgroup john    # Change primary group
usermod -e 2025-12-31 john  # Set account expiry date
usermod -L john             # Lock account
usermod -U john             # Unlock account
usermod -s /bin/zsh john    # Change login shell
usermod -d /new/home -m john # Change home directory and move files
```

Examples:

```
# Add user to sudo group (grant admin privileges)
sudo usermod -aG sudo alice

# Add to multiple groups
sudo usermod -aG developers,docker,www-data alice

# Change user's shell
sudo usermod -s /bin/zsh alice

# Lock user account
sudo usermod -L alice

# Set account expiration
sudo usermod -e 2025-12-31 temp_user
```

ADDITIONAL: User Management Commands

userdel - Delete Users

```
userdel john          # Delete user (keep home directory)
userdel -r john       # Delete user and home directory
userdel -f john       # Force deletion (even if logged in)
```

groupdel - Delete Groups

```
groupdel developers   # Delete group
```

groupmod - Modify Groups

```
groupmod -n newname oldname # Rename group
groupmod -g 2500 groupname  # Change group ID
```

id Command - Display User/Group Information

```
id                # Current user's UID, GID, groups
id username       # Specific user's information
id -u username    # User ID only
id -g username    # Primary group ID only
id -G username    # All group IDs
id -un username   # Username
```

who and w Commands - See Logged In Users

```
who              # Show who is logged in
w               # Show who is logged in and what they're doing
whoami          # Show current username
last            # Show login history
lastlog         # Show most recent login for all users
```

su and sudo Commands - Switch User

```
su                # Switch to root user
su username       # Switch to specific user
su - username     # Switch with user's environment
sudo command      # Execute command as root
sudo -u username command # Execute command as specific user
sudo -i           # Interactive root shell
sudo -l           # List sudo privileges
```

Cybersecurity Best Practices:

```
# Use sudo instead of root login
sudo apt update

# Never work as root user continuously
# Instead, use sudo for privileged commands

# Check sudo access
sudo -l

# View sudo logs
sudo cat /var/log/auth.log | grep sudo

# Audit user accounts
awk -F: '($3 >= 1000) {print $1}' /etc/passwd # Regular users
awk -F: '($3 < 1000) {print $1}' /etc/passwd  # System users
```

chage Command - Password Expiry Management

Purpose: Manages password aging and expiration policies.[^1]

Basic Syntax: `chage [options] username`

Common Uses:

```
chage -l john           # List password info
chage -M 90 john        # Max 90 days before password change required
chage -m 7 john         # Min 7 days between password changes
chage -W 14 john        # Warn 14 days before password expiry
chage -E 2025-12-31 john # Account expires on specific date
chage -d 0 john         # Force password change on next login
chage -I 30 john        # Lock account 30 days after password expiry
chage -E -1 john        # Remove account expiration
```

Examples:

```
# View current settings
sudo chage -l alice

# Set comprehensive password policy
sudo chage -M 90 -m 7 -W 14 -I 30 alice

# Force immediate password change
sudo chage -d 0 alice

# Set account expiration
sudo chage -E 2025-06-30 intern_user
```

8. Input/Output Redirection & Pipes (ADDITIONAL)

Understanding I/O Streams

Linux has three standard data streams:[⁹][8]

- **STDIN (0)** - Standard Input (keyboard)
- **STDOUT (1)** - Standard Output (screen)
- **STDERR (2)** - Standard Error (screen)

Output Redirection

Basic Syntax:

```
command > file           # Redirect output to file (overwrite)
command >> file           # Redirect output to file (append)
command 2> file           # Redirect errors to file
command &> file           # Redirect both output and errors
command 2>&1              # Redirect errors to output stream
command > /dev/null       # Discard output (null device)
command 2> /dev/null      # Discard errors
command &> /dev/null      # Discard everything
```

Examples:

```
# Save command output to file
ls -la > directory_listing.txt

# Append to existing file
echo "New log entry" >> logfile.txt

# Save errors to file
find / -name "*.conf" 2> errors.txt

# Save both output and errors
command &> all_output.txt

# Discard errors (show only output)
find / -name "*.conf" 2>/dev/null

# Save output, discard errors
ls -la /root > output.txt 2>/dev/null
```

Input Redirection

Basic Syntax:

```
command < file           # Use file as input
command << EOF            # Here document (multi-line input)
```



```
command <<< "string"          # Here string (single-line input)
```

Examples:

```
# Use file as input
sort < unsorted.txt

# Create file with here document
cat > config.txt << EOF
server=localhost
port=8080
debug=true
EOF

# Use here string
grep "error" <<< "This is an error message"

# Mail with here document
mail -s "Subject" user@example.com << EOF
This is the email body.
Multiple lines supported.
EOF
```

Pipes - Combining Commands

Purpose: Sends output of one command as input to another command.^{[9][8]}

Basic Syntax: command1 | command2 | command3

Common Pipe Combinations:

```
# Count lines in output
ls -la | wc -l

# Search in output
ps aux | grep firefox

# Sort and filter
cat /etc/passwd | sort | grep "/home"

# Page through long output
ls -lR / | less

# Chain multiple operations
cat access.log | grep "404" | wc -l

# Extract specific fields
cat /etc/passwd | cut -d: -f1,3 | sort

# Find unique values
cat data.txt | sort | uniq
```

```
# Top 10 largest files
du -ah /home | sort -rh | head -n 10
```

Cybersecurity Pipe Examples:

```
# Count failed login attempts
grep "Failed password" /var/log/auth.log | wc -l

# Find most common attacking IPs
grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn

# Monitor active connections
netstat -ant | grep ESTABLISHED | wc -l

# Find processes by memory usage
ps aux | sort -k4 -rn | head -n 10

# Check open ports and services
netstat -tulpn | grep LISTEN | awk '{print $4, $7}'

# Analyze Apache access log for 404 errors
cat access.log | grep " 404 " | awk '{print $7}' | sort | uniq -c | sort -rn
```

ADDITIONAL: Advanced Redirection Techniques

Tee Command - Split Output

```
command | tee file.txt      # Display output AND save to file
command | tee -a file.txt   # Append instead of overwrite

# Examples
ls -la | tee directory.txt  # Show on screen and save
ps aux | tee processes.txt | grep firefox

# Save to multiple files
echo "test" | tee file1.txt file2.txt file3.txt

# Sudo with tee (write to protected files)
echo "new line" | sudo tee -a /etc/hosts
```

Process Substitution

```
# Compare output of two commands
diff <(ls dir1) <(ls dir2)

# Use command output as file
grep "error" <(journalctl -xe)
```

9. Shell Scripting Basics (ADDITIONAL)

What is Shell Scripting?

A shell script is a file containing a series of commands that the shell executes line by line.^[10]^[3]

Why Learn Shell Scripting?

- Automate repetitive tasks
- System administration automation
- Security auditing and monitoring
- Log analysis and reporting
- Backup and maintenance scripts
- Incident response automation^[10]^[3]

Creating Your First Script

Basic Structure:

```
#!/bin/bash
# This is a comment
# Script: hello_world.sh
# Purpose: Print greeting message

echo "Hello, World!"
echo "Current user: $(whoami)"
echo "Current directory: $(pwd)"
```

Steps to Create and Run:

```
# 1. Create script file
nano hello_world.sh

# 2. Add shebang and commands
#!/bin/bash
echo "Hello, World!"

# 3. Save and exit (Ctrl+O, Ctrl+X)

# 4. Make executable
chmod +x hello_world.sh

# 5. Run script
./hello_world.sh
```

Shebang Explained

The `#!` (shebang) line tells system which interpreter to use:[^10]

```
#!/bin/bash          # Bash shell
#!/bin/sh            # System shell
#!/usr/bin/env bash  # Portable bash (recommended)
#!/usr/bin/env python3 # Python script
#!/usr/bin/perl      # Perl script
```

Variables in Shell Scripts

Defining Variables:

```
#!/bin/bash

# Variable assignment (no spaces around =)
NAME="Alice"
AGE=25
TODAY=$(date +%Y-%m-%d)

# Using variables
echo "Name: $NAME"
echo "Age: $AGE"
echo "Today: $TODAY"

# Command substitution
CURRENT_DIR=$(pwd)
FILE_COUNT=$(ls | wc -l)

# Read user input
echo "Enter your name:"
read USERNAME
echo "Hello, $USERNAME!"
```

Special Variables:

```
$0          # Script name
$1, $2, ... # Command-line arguments
 $#         # Number of arguments
$@         # All arguments
$?         # Exit status of last command
$$         # Process ID of script
$!         # Process ID of last background command
```

Example with Arguments:

```
#!/bin/bash
# Script: greet.sh

echo "Script name: $0"
```

```
echo "First argument: $1"
echo "Second argument: $2"
echo "Total arguments: $# "
echo "All arguments: $@"

# Usage: ./greet.sh Alice Bob
```

Conditional Statements

If-Then-Else:

```
#!/bin/bash

if [ $# -eq 0 ]; then
    echo "No arguments provided"
elif [ $# -eq 1 ]; then
    echo "One argument: $1"
else
    echo "Multiple arguments: $@"
fi
```

Comparison Operators:

```
# Numeric comparisons
-eq      # Equal
-ne      # Not equal
-gt      # Greater than
-lt      # Less than
-ge      # Greater or equal
-le      # Less or equal

# String comparisons
=        # Equal
!=       # Not equal
-z       # Empty string
-n       # Non-empty string

# File tests
-f       # File exists
-d       # Directory exists
-r       # Readable
-w       # Writable
-x       # Executable
-s       # Non-empty file
```

Examples:

```
#!/bin/bash

# Check if file exists
if [ -f "/etc/passwd" ]; then
    echo "File exists"
```

```

fi

# Check if directory exists
if [ -d "/home/$USER" ]; then
    echo "Home directory exists"
fi

# Numeric comparison
AGE=25
if [ $AGE -ge 18 ]; then
    echo "Adult"
else
    echo "Minor"
fi

# String comparison
NAME="Alice"
if [ "$NAME" = "Alice" ]; then
    echo "Welcome, Alice!"
fi

# Check if root user
if [ $(id -u) -eq 0 ]; then
    echo "Running as root"
else
    echo "Not root user"
fi

```

Loops

For Loop:

```

#!/bin/bash

# Loop through numbers
for i in {1..5}; do
    echo "Number: $i"
done

# Loop through strings
for COLOR in red green blue; do
    echo "Color: $COLOR"
done

# Loop through files
for FILE in *.txt; do
    echo "Processing: $FILE"
done

# C-style for loop
for ((i=1; i<=10; i++)); do
    echo "Count: $i"
done

```

While Loop:

```
#!/bin/bash

# While loop
COUNT=1
while [ $COUNT -le 5 ]; do
    echo "Iteration: $COUNT"
    ((COUNT++))
done

# Read file line by line
while read LINE; do
    echo "Line: $LINE"
done < input.txt

# Infinite loop (Ctrl+C to stop)
while true; do
    echo "Running..."
    sleep 1
done
```

Until Loop:

```
#!/bin/bash

COUNT=1
until [ $COUNT -gt 5 ]; do
    echo "Count: $COUNT"
    ((COUNT++))
done
```

Functions

Defining and Using Functions:

```
#!/bin/bash

# Define function
greet() {
    echo "Hello, $1!"
}

# Call function
greet "Alice"
greet "Bob"

# Function with return value
add_numbers() {
    local SUM=$(( $1 + $2 ))
    echo $SUM
}
```

```

RESULT=$(add_numbers 5 10)
echo "Sum: $RESULT"

# Function with multiple parameters
create_backup() {
    local SOURCE=$1
    local DEST=$2
    echo "Backing up $SOURCE to $DEST"
    tar -czf "$DEST/backup_$(date +%Y%m%d).tar.gz" "$SOURCE"
}

create_backup "/etc" "/backup"

```

Practical Cybersecurity Scripts

System Monitoring Script:

```

#!/bin/bash
# Script: system_monitor.sh
# Purpose: Monitor system resources

echo "==== System Monitor ====="
echo "Date: $(date)"
echo ""

echo "CPU Usage:"
top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d% -f1

echo ""
echo "Memory Usage:"
free -h | grep Mem | awk '{print "Total: "$2 " Used: "$3 " Free: "$4}'

echo ""
echo "Disk Usage:"
df -h / | tail -1 | awk '{print "Used: "$3 " / "$2 (" $5")"}'

echo ""
echo "Active Connections:"
netstat -ant | grep ESTABLISHED | wc -l

```

Log Analysis Script:

```

#!/bin/bash
# Script: analyze_auth_log.sh
# Purpose: Analyze authentication attempts

LOG_FILE="/var/log/auth.log"

echo "==== Authentication Analysis ====="
echo ""

echo "Failed Login Attempts:"

```



```

FAILED=$(grep "Failed password" $LOG_FILE | wc -l)
echo "Total: $FAILED"

echo ""
echo "Top 5 Attacking IPs:"
grep "Failed password" $LOG_FILE | \
    awk '{print $11}' | \
    sort | uniq -c | sort -rn | head -5

echo ""
echo "Successful Logins:"
grep "Accepted password" $LOG_FILE | wc -l

```

Backup Script:

```

#!/bin/bash
# Script: backup.sh
# Purpose: Automated backup system

SOURCE="/home/user/important"
DEST="/backup"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="backup_${DATE}.tar.gz"

echo "Starting backup..."
tar -czf "$DEST/$BACKUP_FILE" "$SOURCE" 2>/dev/null

if [ $? -eq 0 ]; then
    echo "Backup successful: $BACKUP_FILE"

    # Delete backups older than 7 days
    find $DEST -name "backup_*.tar.gz" -mtime +7 -delete
    echo "Old backups cleaned"
else
    echo "Backup failed!"
    exit 1
fi

```

10. Environment Variables (ADDITIONAL)

Understanding Environment Variables

Environment variables store system and user configuration information accessible to all processes.^{[11][12]}

Common Environment Variables:

\$HOME	# User's home directory
\$USER	# Current username
\$SHELL	# Default shell
\$PATH	# Command search path

```
$PWD          # Current working directory
$OLDPWD       # Previous working directory
$LANG        # System language
$EDITOR       # Default text editor
$HOSTNAME     # System hostname
```

Viewing Environment Variables:

```
env           # List all environment variables
printenv      # Same as env
echo $HOME    # Display specific variable
echo $PATH    # Display PATH
export        # Show exported variables
```

PATH Variable

The PATH variable specifies directories where the shell looks for executable commands.^[12]^[11]

Viewing PATH:

```
echo $PATH
# Output: /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

Adding to PATH (Temporary):

```
export PATH=$PATH:/new/directory
export PATH=$PATH:~/scripts
```

Adding to PATH (Permanent):

```
# Edit .bashrc or .profile
nano ~/.bashrc

# Add at end
export PATH=$PATH:~/bin
export PATH=$PATH:/opt/custom/bin

# Reload configuration
source ~/.bashrc
```

Setting Custom Environment Variables:

```
# Temporary (current session only)
export MY_VAR="value"

# Permanent (add to ~/.bashrc)
echo 'export MY_VAR="value"' >> ~/.bashrc
source ~/.bashrc
```

```
# System-wide (requires root)
sudo nano /etc/environment
# Add: MY_VAR="value"
```

Common Configuration Files:

- **~/.bashrc** - Bash configuration (interactive non-login shells)
- **~/.bash_profile** - Bash login shells
- **~/.profile** - Generic profile (all shells)
- **/etc/environment** - System-wide variables
- **/etc/profile** - System-wide login configuration^[11]

11. Networking Commands (4:38:50 - 6:05:18)

ping Command - Test Connectivity

Purpose: Tests network connectivity to remote hosts by sending ICMP echo requests.[^1]

Basic Syntax: `ping [options] host`

Common Uses:

```
ping google.com           # Continuous ping (Ctrl+C to stop)
ping -c 4 google.com      # Send 4 packets only
ping -c 10 8.8.8.8        # Ping Google DNS
ping -i 2 google.com      # Interval of 2 seconds between packets
ping -s 1000 google.com   # Packet size of 1000 bytes
ping -W 5 google.com      # Timeout of 5 seconds
ping -q -c 10 google.com  # Quiet mode (summary only)
```

Output Interpretation:

[illegible]

Key Metrics:

- **Time (latency):** Lower is better (< 50ms excellent, 50-100ms good, > 100ms slow)
- **Packet loss:** Should be 0% (any loss indicates network issues)

- **TTL (Time To Live):** Number of hops remaining (typically starts at 64, 128, or 255)[^1]

Cybersecurity Applications:

```
# Test if host is reachable
ping -c 4 target.com

# Detect if host is blocking ICMP
ping -c 4 suspicious-host.com
# No response could mean firewall blocking ICMP

# Network latency testing
ping -c 100 server.com | tail -3

# Flood ping (stress test - requires root, use ethically!)
sudo ping -f target.com
```

traceroute Command - Trace Network Path

Purpose: Shows the route packets take to reach destination, displaying each hop (router).[^1]

Basic Syntax: `traceroute [options] host`

Installation (if not available):

```
sudo apt install traceroute
```

Common Uses:

```
traceroute google.com
traceroute -m 20 example.com # Max 20 hops
traceroute -n 8.8.8.8        # Don't resolve hostnames (faster)
traceroute -q 5 google.com   # 5 queries per hop
```

Understanding Output:

```
traceroute to google.com (142.250.185.46), 30 hops max, 60 byte packets
1  router.local (192.168.1.1)  1.234 ms  1.123 ms  1.089 ms
2  10.20.30.1 (10.20.30.1)    5.432 ms  5.398 ms  5.367 ms
3  * * *
4  142.250.185.46 (142.250.185.46) 12.456 ms 12.432 ms 12.398 ms
```

Explanation:

- Hop number
- Router hostname/IP
- Three time measurements (

[^13][^14][^15][^16][^17][^18][^19][^20][^21]

✖

[^1]: course.pdf

[^2]: <https://www.geeksforgeeks.org/linux-unix/linux-file-hierarchy-structure/>
[^3]: <https://www.geeksforgeeks.org/linux-unix/introduction-linux-shell-shell-scripting/>
[^4]: <https://www.webasha.com/blog/exploring-linux-package-managers-understanding-apt-yun>
[^5]: https://en.wikipedia.org/wiki/Filesystem_Hierarchy_Standard
[^6]: <https://www.cherryservers.com/blog/a-complete-guide-to-understanding-linux-file-sys>
[^7]: <https://www.linuxjournal.com/content/linux-file-system-hierarchy-your-guide-understa>
[^8]: <https://www.redhat.com/en/blog/linux-shell-redirection-pipelining>
[^9]: <https://www.tutorialspoint.com/pipes-and-redirection-in-linux>
[^10]: <https://www.freecodecamp.org/news/shell-scripting-crash-course-how-to-write-bash-s>
[^11]: <https://stackoverflow.com/questions/37676849/where-is-path-variable-set-in-ubuntu>
[^12]: https://www.ibm.com/docs/ssw_aix_72/security/user_accounts_path_env_variable.html
[^13]: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/4/html/reference
[^14]: <https://labex.io/lesson/filesystem-hierarchy>
[^15]: <https://systemd.io>
[^16]: <https://www.freecodecamp.org/news/a-brief-overview-and-history-of-systemd-the-linu>
[^17]: <https://www.redhat.com/en/blog/how-manage-packages>
[^18]: <https://www.shellscript.sh>
[^19]: <https://www.youtube.com/watch?v=RvYKa5S6BRY>
[^20]: <https://www.coursera.org/learn/hands-on-introduction-to-linux-commands-and-shell-s>
[^21]: https://www.tutorialspoint.com/unix/shell_scripting.htm